Alumno. Sebastián González Juárez

Práctica de laboratorio 3

1. **Bernstein-Vazirani**

(a) (12.5 pts.) Demostrar la identidad de las matrices Hadamards

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x\cdot z}\,|z\rangle$$

Usando la formula anterior, explicarme en tus propias palabras como aplicar n compuertas Hadamard a los primeros n qubits nos recupera el ket s en la operación.

$$|\psi_3\rangle = H^{\otimes n}|\psi_2\rangle = H^{\otimes n}\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{x\cdot z}\,|x\rangle = |s\rangle$$

Solución.

Sean $x = x_1x_2\cdots x_n$ y $z = z_1z_2\cdots z_3$ cadenas binarias de longitud n, y $x\cdot z = x_1z_1 \oplus \cdots \oplus x_nz_n$ es el producto punto módulo 2. La compuerta Hadamard sobre un qubit actúa como: $H|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$, $H|1\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$

Estas dos expresiones se pueden escribir como:

$$H|x_i\rangle = \frac{1}{\sqrt{2}} \sum_{z_i\in\{0,1\}} (-1)^{x_iz_i}\,|z_i\rangle, \qquad x_i, z_i \in \{0,1\}, \qquad \text{vemos que:} \begin{cases} \text{si } x_i = 0 \Rightarrow (-1)^{0\cdot z_i} = 1 \Rightarrow H|0\rangle \\ \text{si } x_i = 1 \Rightarrow (-1)^{1\cdot z_i} = (-1)^{z_i} \Rightarrow H|1\rangle \end{cases}$$

Ahora aplicamos $H^{\otimes n}$ al estado $|x\rangle = |x_1\rangle \otimes |x_2\rangle \otimes \cdots \otimes |x_n\rangle$: $H^{\otimes n}|x\rangle = H|x_1\rangle \otimes H|x_2\rangle \otimes \cdots \otimes H|x_n\rangle$

Sustituyendo la expresión anterior para cada qubit,

$$\begin{aligned} H^{\otimes n}|x\rangle &= \left(\frac{1}{\sqrt{2}} \sum_{z_1\in\{0,1\}} (-1)^{x_1z_1}\,|z_1\rangle\right) \otimes \cdots \otimes \left(\frac{1}{\sqrt{2}} \sum_{z_n\in\{0,1\}} (-1)^{x_nz_n}\,|z_n\rangle\right) \\ &= \left(\frac{1}{\sqrt{2}}\right)^n \sum_{z_1,\dots,z_n\in\{0,1\}} (-1)^{x_1z_1}\cdots(-1)^{x_nz_n}|z_1\cdots z_n\rangle = \frac{1}{\sqrt{2^n}} \sum_{z_1,\dots,z_n\in\{0,1\}} (-1)^{x_1z_1+\cdots+x_nz_n}\,|z_1\cdots z_n\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}} (-1)^{x\cdot z}\,|z\rangle \qquad \blacksquare \end{aligned}$$

En Bernstein–Vazirani, después del oráculo el primer registro queda en el estado:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{x\cdot s}\,|x\rangle$$

Al aplicar la Hadamard a los primeros n qubits y utilizando lo que hemos demostrado:

$$|\psi_3\rangle = H^{\otimes n}|\psi_2\rangle = H^{\otimes n}\left(\frac{1}{\sqrt{2^n}}\sum_x (-1)^{x\cdot s}\,|x\rangle\right) = \frac{1}{\sqrt{2^n}}\sum_x (-1)^{x\cdot s}\,H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}}\sum_x (-1)^{x\cdot s}\left(\frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}} (-1)^{x\cdot z}\,|z\rangle\right)$$

Entonces,

$$|\psi_3\rangle = \frac{1}{2^n}\sum_z \left(\sum_x (-1)^{x\cdot s}(-1)^{x\cdot z}\right)|z\rangle = \frac{1}{2^n}\sum_z \left(\sum_x (-1)^{x\cdot(s\oplus z)}\right)|z\rangle$$

- Si $z = s$, entonces $s \oplus z = 0$, y por tanto $(-1)^{x\cdot 0} = 1$ para toda x. En ese caso la suma interior vale 2^n.
- Si $z \neq s$, entonces $s \oplus z \neq 0$, y la mitad de los términos de la suma valen $+1$ y la otra mitad valen -1, así que todas las contribuciones se cancelan y la suma da 0, dando interferencia constructiva.

Por eso el estado final queda exactamente en $|s\rangle$, y al medir recuperamos directamente el string oculto.

(b) (12.5 pts.) ¿Es el oráculo de Bernstein-Vazirani igual a aplicar una función balanceada en el algoritmo de Deutsch-Jozsa? ¿Si, no, por qué?

Respuesta.

No es para nada lo mismo, pues, aunque existe una relación entre ambos, en el algoritmo de Deutsch–Jozsa, el objetivo es determinar si una función $f: \{0,1\}^n \to \{0,1\}$ es constante o balanceada (sin importar su forma específica) en cambio, Bernstein–Vazirani el oráculo corresponde a una función particular de la forma $f_s(x) = s \cdot x (\mathrm{mod} 2)$, donde s es un string oculto que se desea encontrar.

Si $s \neq 0^n$, esta función resulta ser balanceada, ya que produce 0 para la mitad de los valores de x y 1 para la otra mitad; sin embargo, el algoritmo de Bernstein–Vazirani no se limita a clasificar la función, más bien, aprovecha su estructura lineal para recuperar completamente el string s en una sola ejecución ideal. Por lo tanto, aunque el oráculo de Bernstein–Vazirani puede verse como un caso particular de función balanceada, ambos algoritmos no son equivalentes, ya que resuelven problemas distintos.

(c) (12.5 pts.) Analiza cómo funciona la función bv_oracle(n, s) y explícame su funcionamiento en tus propias palabras con el circuito de Bernstein-Vazirani. Programa el algoritmo de Bernstein–Vazirani SIN USAR LA FUNCIÓN bv_oracle(n, s) para los siguientes strings, explicando en el código lo que hiciste: s = 0110, s = 1101, s = 1111, s = 1010.

Nota.

El desarrollo del código y solución del ejercicio viene al final del documento anexado como pdf, pero desarrollado en Google colab con Qiskit.

2. Simon

(a) (12.5 pts.)

Demuestra que si tenemos una función 2-a-1 del tipo que Simon estudia, de modo que $f(x) = f(x \oplus s)$ para algún string s dado, entonces dados x y y tales que $f(x) = f(y),\ x \neq y$, entonces $x \oplus y = s$ y explica como usamos esta propiedad para resolver el problema de Simon de forma clásica.

Solución.

Sea $f: \{0,1\}^n \to \{0,1\}^m$ una función del tipo que estudia Simon, es decir, una función 2-a-1 para la cual existe un string fijo $s \in \{0,1\}^n$, con $s \neq 0^n$, tal que $f(x) = f(x \oplus s),\ \forall x \in \{0,1\}^n$.

Por definición de f, cada valor de salida tiene exactamente dos preimágenes, y dichas dos preimágenes difieren en el string oculto s. En otras palabras, para cada x, las dos entradas que producen la misma salida son x y $x \oplus s$. Por tanto, si x y y satisfacen $f(x) = f(y), x \neq y$, entonces necesariamente $y = x \oplus s$.

$$\Rightarrow x \oplus y = x \oplus (x \oplus s) = (x \oplus x) \oplus s = 0 \oplus s = s, \qquad \therefore x \oplus y = s \quad \blacksquare$$

Esta propiedad permite resolver el problema de Simon de manera clásica evaluando la función en distintas entradas y buscando dos valores distintos $x \neq y$ tales que $f(x) = f(y)$; cuando se encuentra una colisión, entonces se recupera automáticamente el string oculto mediante $s = x \oplus y$. Sin embargo, este procedimiento clásico es ineficiente, ya que en general requiere evaluar la función muchas veces hasta encontrar una colisión, lo cual crece exponencialmente con n.

Demuestra que $(z \oplus s) \cdot y = (z \cdot y) \oplus (s \cdot y)$ y con eso completa la parte de la demostración de Simon que faltaba en el notebook y explica porque los strings y que recuperas cumplen que $s \cdot y = 0$ cuando corres el algoritmo de Simon y lees el primer registro. Las definiciones de $(x \oplus y$ y $x \cdot y)$ se encuentran en el notebook.

Solución.

Sean $z,\ s,\ y \in \{0,1\}^n$. Entonces, por definición del producto punto módulo 2:

$$\begin{aligned}(z \oplus s) \cdot y &= (z_1 \oplus s_1)y_1 \oplus (z_2 \oplus s_2)y_2 \oplus \cdots \oplus (z_n \oplus s_n)y_n = (z_1y_1 \oplus s_1y_1) \oplus \cdots \oplus (z_ny_n \oplus s_ny_n) \\ &= (z_1y_1 \oplus \cdots \oplus z_ny_n) \oplus (s_1y_1 \oplus \cdots \oplus s_ny_n) = (z \cdot y) \oplus (s \cdot y) \qquad \blacksquare\end{aligned}$$

En el algoritmo de Simon, después de aplicar el oráculo y medir el segundo registro, el primer registro colapsa a un estado de la forma: $|\phi\rangle = \frac{1}{\sqrt{2}}(|z\rangle + |z \oplus s\rangle)$ para algún $z \in \{0,1\}^n$. Luego se aplica $H^{\otimes n}$ al primer registro.

$$\begin{aligned}H^{\otimes n}|x\rangle &= \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y}\, |y\rangle \\ \Rightarrow H^{\otimes n}|\phi\rangle &= \frac{1}{\sqrt{2}}\left(H^{\otimes n}|z\rangle + H^{\otimes n}|z \oplus s\rangle\right) = \frac{1}{\sqrt{2^{n+1}}} \sum_{y \in \{0,1\}^n} \left[(-1)^{z \cdot y} + (-1)^{(z \oplus s) \cdot y}\right] |y\rangle\end{aligned}$$

Ahora, usando la identidad ya demostrada,

$$\begin{aligned}(z \oplus s) \cdot y = (z \cdot y) \oplus (s \cdot y) &\Rightarrow (-1)^{(z \oplus s) \cdot y} = (-1)^{(z \cdot y) \oplus (s \cdot y)} = (-1)^{z \cdot y}(-1)^{s \cdot y} \\ &\Rightarrow (-1)^{z \cdot y} + (-1)^{(z \oplus s) \cdot y} = (-1)^{z \cdot y}(1 + (-1)^{s \cdot y})\end{aligned}$$

$\therefore$ Se concluye que la amplitud del estado $|y\rangle$ es proporcional a $(1 + (-1)^{s \cdot y})$.

Si $s \cdot y = 0$, entonces este factor vale 2 y la amplitud sobrevive; en cambio, si $s \cdot y = 1$, el factor vale 0 y la amplitud se cancela. Por lo tanto, al medir el primer registro solo pueden obtenerse strings $y$que satisfacen $s \cdot y = 0$, es decir, strings ortogonales a $s$en sentido módulo 2. Esto explica por qué cada ejecución del algoritmo de Simon produce una ecuación lineal homogénea sobre el string oculto s, y por qué al reunir suficientes ecuaciones independientes es posible determinarlo completamente.

(b) (12.5 pts.) Piensa en 3 funciones 2-a-1 para los siguientes 3 strings: s = 1010, s = 1101, s = 1111 y fabrica el oráculo de estas (Recuerda que CNOT (x, y) = (x, y ⊕ x)). Recupera los strings utilizando Simon y resuelve el sistema de ecuaciones con el método que tú quieras. No olvides que Qiskit invierte el orden de los strings.

Nota.

La solución del ejercicio se acompaña con el código que viene al final del documento anexado como pdf, pero desarrollado en Google colab con Qiskit. (Al inicio agrego funciones de utilidades que son ocupadas)

Solución.

Para este inciso se construyeron tres funciones 2-a-1, una para cada string oculto pedido: $s = 1010$, $s = 1101$, $s = 1111$. La idea fue definir funciones del tipo $f(x) = Mx$ (mod2), donde M es una matriz binaria de rango 3. De esta forma, el núcleo de la transformación tiene exactamente dos elementos, 0000 y s, por lo que se cumple $f(x) = f(x \oplus s)$, y la función resulta ser 2-a-1, como requiere el problema de Simon.

El oráculo cuántico se construyó usando la regla $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$, de modo que cada suma módulo 2 en la salida se implementó con compuertas CNOT, aprovechando que $\mathrm{CNOT}(x, y) = (x, y \oplus x)$. Así, cada bit de salida se fabricó como combinación XOR de ciertos bits de entrada.

Resultados:

```
--- Caso s = 1010 ---
Mediciones acumuladas: {'1011': 3, '1111': 1, '0000': 1, '0101': 1}
Ecuaciones independientes (y):
  1011
  1111
  0000
  0101
Candidatos para s: ['1010']
String recuperado: 1010
Rango del sistema: 3

--- Caso s = 1101 ---
Mediciones acumuladas: {'1001': 2, '0000': 1, '0101': 2, '0111': 1}
Ecuaciones independientes (y):
  1001
  0000
  0101
  0111
Candidatos para s: ['1101']
String recuperado: 1101
Rango del sistema: 3

--- Caso s = 1111 ---
Mediciones acumuladas: {'1010': 1, '1100': 1, '1001': 1}
Ecuaciones independientes (y):
  1010
  1100
  1001
Candidatos para s: ['1111']
String recuperado: 1111
Rango del sistema: 3
```

- Caso 1: $s = 1010$

Para este caso se construyó una función 2-a-1 tal que las entradas x y $x \oplus 1010$ producen la misma salida, cumpliendo así la condición central del problema de Simon. Al ejecutar el algoritmo, las mediciones del primer registro dieron los strings 1011, 1111, 0000 y 0101. Cada uno de estos resultados satisface la relación $y \cdot s = 0(\mathrm{mod}2)$, por lo que pueden interpretarse como ecuaciones lineales sobre el string oculto. Al resolver el sistema en $\mathbb{F}_2$, el único candidato no trivial que satisface simultáneamente todas las ecuaciones es $s = 1010$. El rango del sistema fue 3, lo cual es suficiente para determinar unívocamente el periodo oculto. Por tanto, el algoritmo recuperó correctamente el string buscado.

- Caso 2: $s = 1101$

En este caso se fabricó un oráculo 2-a-1 con la propiedad $f(x) = f(x \oplus 1101)$, de modo que el periodo oculto de la función fuera precisamente 1101. Al correr Simon se obtuvieron como ecuaciones independientes los strings 1001, 0000, 0101 y 0111. Todos ellos cumplen la condición de ortogonalidad módulo 2 con el string

secreto, es decir, $y \cdot s = 0(\text{mod}2)$. A partir de estas ecuaciones se formó el sistema lineal correspondiente y, al resolverlo, se obtuvo como único candidato no trivial $s = 1101$. De nuevo, el rango del sistema fue 3, indicando que se reunió la información suficiente para identificar de manera única el string oculto. Así, también en este caso el algoritmo de Simon recuperó correctamente el valor esperado.

- Caso 3: $s = 1111$

Para el string $s = 1111$ se construyó igualmente una función 2-a-1 que cumple $f(x) = f(x \oplus 1111)$, asegurando que las entradas emparejadas por ese periodo tengan la misma imagen. Al ejecutar el algoritmo se obtuvieron las ecuaciones 1010, 1100 y 1001, todas consistentes con la condición $y \cdot s = 0(\text{mod}2)$. Estas ecuaciones generan un sistema lineal de rango 3, suficiente para restringir la solución a un único candidato no trivial. Al resolverlo en $\mathbb{F}_2$, se recuperó el string $s = 1111$, coincidiendo exactamente con el valor usado para construir el oráculo. Por lo tanto, el procedimiento funcionó correctamente también en este tercer caso, confirmando que Simon permite reconstruir el periodo oculto a partir de las mediciones del primer registro.

(c) (12.5 pts.) Explica que hace la función simon_oracle(s) del notebook y por qué siempre nos recupera un oráculo 2-a-1 y que hace si el string s solo tiene 0. Explícalo en tus propias palabras y haz la demostración matemática.

Solución.

La función simon_oracle(s) construye un circuito cuántico que implementa una función $f(x)$ con la propiedad característica del problema de Simon, es decir, una función 2-a-1 tal que $f(x) = f(x \oplus s)$, donde $s$es el string oculto. En términos del circuito, el oráculo actúa como $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$, por lo que la salida se obtiene aplicando operaciones XOR (implementadas con compuertas CNOT) entre el registro de entrada y el de salida. La construcción del oráculo está diseñada de tal forma que ciertos bits de salida dependen linealmente de los bits de entrada, y cuando un bit de s es igual a 1, se introducen correlaciones que hacen que dos entradas distintas x y $x \oplus s$ produzcan exactamente la misma salida.

Para ver por qué siempre se obtiene una función 2-a-1, consideremos que la función implementada por el oráculo es lineal módulo 2, es decir, $f(x) = Mx(\text{mod}2)$, donde M es una matriz binaria. La construcción del oráculo garantiza que $s$pertenece al kernel de M, es decir, $Ms = 0$. Entonces, para cualquier x,

$$f(x \oplus s) = M(x \oplus s) = Mx \oplus Ms = Mx,$$

ya que $Ms = 0$. Esto implica que $f(x) = f(x \oplus s)$, por lo que cada salida tiene exactamente dos preimágenes: xy $x \oplus s$. Además, si $x \neq x \oplus s$(lo cual ocurre siempre que $s \neq 0$), entonces la función es estrictamente 2-a-1.

Por otro lado, si el string s es el string nulo $s = 00 \cdots 0$, entonces se cumple que $x \oplus s = x$, por lo que la condición anterior se vuelve trivial: $f(x) = f(x)$.

En este caso ya no hay pares distintos que compartan la misma imagen, y la función deja de ser 2-a-1 para convertirse en una función 1-a-1 (inyectiva). Esto implica que cada entrada tiene una salida distinta, y el algoritmo de Simon no puede encontrar un periodo oculto no trivial, ya que dicho periodo simplemente no existe.

(d) (12.5 pts.) Genera 2 oráculos de funciones 1-a-1 y corrobora que Simon nos regresa todos los strings con igual probabilidad, y compara con 2 oráculos de funciones 2-a-1 (puedes usar los que ya están en el notebook). Puedes hacerlo de la forma que tú quieras, pero te recomiendo que hagas histogramas de las mediciones que recuperaste para comparar los datos de las mediciones. Coloca el código del algoritmo, del oráculo y de todas las gráficas de forma ordenada y clara.

Nota. El desarrollo del código y solución del ejercicio viene al final del documento anexado como pdf, pero desarrollado en Google colab con Qiskit. Ahí viene la conclusión.

3. (12.5 pts.) Investiga qué significa el término Noisy Intermediate-Scale Quantum. Con ello, contesta las siguientes preguntas:

El término Noisy Intermediate-Scale Quantum (NISQ) fue introducido por John Preskill para describir la etapa actual de la computación cuántica, caracterizada por dispositivos con un número intermedio de qubits (del orden de decenas a cientos) pero aún limitados por la presencia de ruido y la ausencia de corrección de errores a gran escala. En esta era, las computadoras cuánticas son lo suficientemente grandes como para explorar problemas no triviales, pero no lo suficientemente robustas como para ejecutar algoritmos largos de forma confiable.

(a) ¿Qué caracteriza a las computadoras cuánticas de la era NISQ?

Las computadoras NISQ se caracterizan por combinar un número creciente de qubits con limitaciones físicas importantes. En particular, presentan errores en compuertas de uno y dos qubits, tiempos de coherencia finitos y errores de medición, lo que restringe la profundidad efectiva de los circuitos que pueden ejecutarse. Además, el rendimiento de los algoritmos depende fuertemente de factores como la conectividad entre qubits y la fidelidad de las operaciones, lo cual ha sido evidenciado en implementaciones recientes donde la arquitectura del hardware impacta directamente el resultado final del algoritmo.

(b) ¿Por qué el ruido y la decoherencia el funcionamiento de los algoritmos cuánticos?

El funcionamiento de los algoritmos cuánticos depende críticamente de la preservación de la superposición, la fase relativa y la interferencia cuántica. La decoherencia, causada por la interacción del sistema con el entorno, degrada estas propiedades y provoca que el estado cuántico evolucione hacia una mezcla clásica. Además, los errores de compuerta se acumulan a medida que aumenta la profundidad del circuito, lo que provoca desviaciones significativas respecto al resultado ideal. En particular, las compuertas de dos qubits son especialmente propensas a errores, y su uso intensivo en circuitos complejos puede deteriorar rápidamente la fidelidad del algoritmo, como se ha observado en implementaciones experimentales.

(c) Considerando el número de qubits y las compuertas que utilizan ¿Que tan factible es correr los algoritmos de Bernstein-Vazirano y de Simon en computadoras NISQ?

El algoritmo de Bernstein-Vazirani es considerablemente más factible de implementar en dispositivos NISQ que el algoritmo de Simon. Bernstein-Vazirani requiere un número relativamente pequeño de qubits ($n + 1$ en su forma estándar, o incluso n en versiones optimizadas) y solo una consulta al oráculo para determinar la cadena oculta. Además, su implementación puede construirse con un número reducido de compuertas y una baja dependencia del entrelazamiento, lo que lo hace menos sensible al ruido.

En contraste, el algoritmo de Simon requiere aproximadamente $2n$ qubits y múltiples ejecuciones del circuito para obtener suficientes ecuaciones lineales que permitan reconstruir la cadena secreta. Esto implica una mayor profundidad efectiva y una acumulación de errores más significativa. Implementaciones reales en hardware superconductivo han mostrado que el desempeño del algoritmo se degrada rápidamente con el tamaño del problema, debido tanto a errores de compuerta como a limitaciones en la conectividad física entre qubits. Por lo tanto, mientras Bernstein-Vazirani es viable en instancias pequeñas y medianas en hardware NISQ, Simon queda restringido a demostraciones de pequeña escala.

(d) ¿Cuál de los dos algoritmos crees que sería más robusto frente al ruido y por qué?

El algoritmo de Bernstein-Vazirani es más robusto frente al ruido en comparación con el algoritmo de Simon. Esto se debe a que requiere menos qubits, menor profundidad de circuito y una sola ejecución para obtener la solución. Además, su implementación puede evitar en gran medida el uso intensivo de entrelazamiento, lo que reduce su sensibilidad a errores en compuertas de dos qubits. Por el contrario, el algoritmo de Simon depende de múltiples ejecuciones y de la acumulación de resultados correctos para reconstruir la solución, lo que lo hace altamente

vulnerable a errores experimentales. Estudios en hardware real han mostrado que el ruido puede generar resultados inválidos (vectores que no satisfacen la condición de ortogonalidad esperada) y aumentar significativamente la tasa de error del algoritmo.

(e) Investiga un experimento real donde alguno de estos algoritmos haya sido implementado en hardware cuántico y describe brevemente el resultado.

Un ejemplo experimental relevante es la implementación del algoritmo de Bernstein-Vazirani en sistemas de iones atrapados. En este trabajo, los autores implementaron el algoritmo utilizando dos y tres qubits físicos, logrando identificar correctamente la cadena oculta con probabilidades de éxito en el caso de dos qubits y en el caso de tres qubits. Estos resultados demuestran que el algoritmo puede ejecutarse con alta fidelidad en sistemas físicos reales de pequeña escala.

Otro ejemplo más reciente es el trabajo de Pokharel y Lidar, quienes implementaron una versión del algoritmo de Bernstein-Vazirani en procesadores superconductores de IBM de 27 qubits. En este caso, los autores reportaron una demostración de ventaja algorítmica cuántica, mostrando que el algoritmo supera a su contraparte clásica bajo ciertas condiciones, especialmente cuando se emplean técnicas de mitigación de ruido como el dynamical decoupling. Este resultado es particularmente relevante porque evidencia que, incluso en la era NISQ, es posible observar beneficios cuánticos reales, aunque estos dependen fuertemente de la calidad del hardware y del control del ruido.

En conjunto, estos resultados muestran que los algoritmos cuánticos pueden ejecutarse en dispositivos NISQ, pero su desempeño está fuertemente condicionado por las limitaciones físicas del hardware, siendo los algoritmos más simples y de menor profundidad (como Bernstein-Vazirani) los más adecuados para esta etapa tecnológica.

Referencias:

J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.

A. Kandala et al., “Error mitigation extends the computational reach of a noisy quantum processor,” *Nature*, vol. 567, pp. 491–495, 2019.

M. A. Nielsen and I. L. Chuang, “Quantum Computation and Quantum Information,” Cambridge University Press, 2000.

E. Bernstein and U. Vazirani, “Quantum complexity theory,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1411–1473, 1997.

D. R. Simon, “On the power of quantum computation,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1474–1483, 1997.

Y. Debnath et al., “Demonstration of a small programmable quantum computer with atomic qubits,” *Nature*, vol. 536, pp. 63–66, 2016.

B. Pokharel and D. A. Lidar, “Demonstration of Algorithmic Quantum Speedup,” *Physical Review Letters*, vol. 130, 210602, 2023.

Computación Cuántica II

Sebastián González Juárez

Práctica de laboratorio 3.

1. Bernstein-Vazirani (C)

```
!pip install qiskit qiskit-aer pylatexenc matplotlib
```

Mostrar salida oculta

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

Construcción del circuito sin usar bv_oracle(n,s).

```
def bernstein_vazirani_manual(s):
    n = len(s)
    qc = QuantumCircuit(n + 1, n)

    # 1) Preparar el qubit auxiliar en |1>
    qc.x(n)

    # 2) Aplicar Hadamard a todos los qubits
    qc.h(range(n + 1))

    # 3) Construir manualmente el oráculo (Se recorre s al revés por el orden c
    s_reversa = s[::-1]
    for i, bit in enumerate(s_reversa):
        if bit == "1":
            qc.cx(i, n)

    # 4) Aplicar Hadamard otra vez al registro de entrada
    qc.h(range(n))

    # 5) Medir el registro de entrada
    qc.measure(range(n), range(n))

    return qc
```

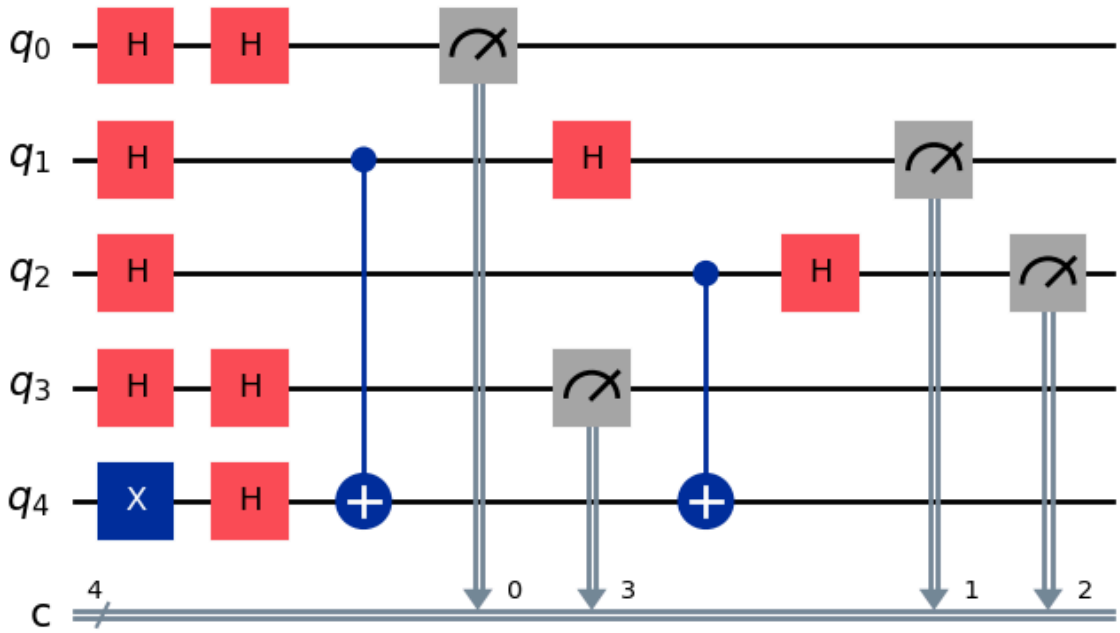
Función para dibujar y ejecutar:

```
def ejecutar_caso_bv(s, shots=1024):
    simulator = AerSimulator()

    # Construcción del circuito
    qc = bernstein_vazirani_manual(s)

    # Mostrar circuito
    print(f"\nCircuito para s = {s}")
    display(qc.draw(output="mpl"))

    # Ejecución
    tqc = transpile(qc, simulator)
    result = simulator.run(tqc, shots=shots).result()
    counts = result.get_counts()

    # Mostrar resultados
    print(f"Resultados para s = {s}: {counts}")
    display(plot_histogram(counts))

    return qc, counts
```

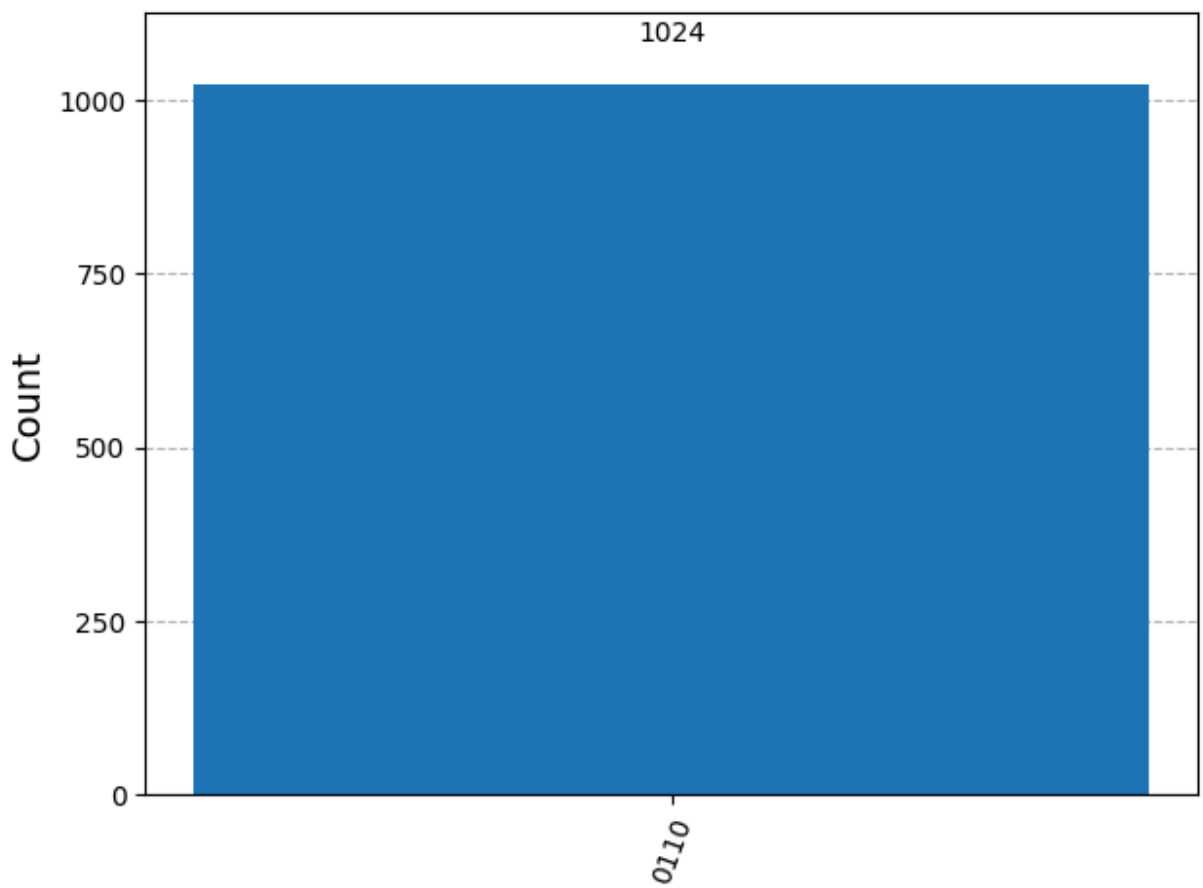
Caso: s=0110

```
qc_0110, counts_0110 = ejecutar_caso_bv("0110", shots=1024)
```

```
Circuito para s = 0110
```




```
Resultados para s = 0110: {'0110': 1024}
```

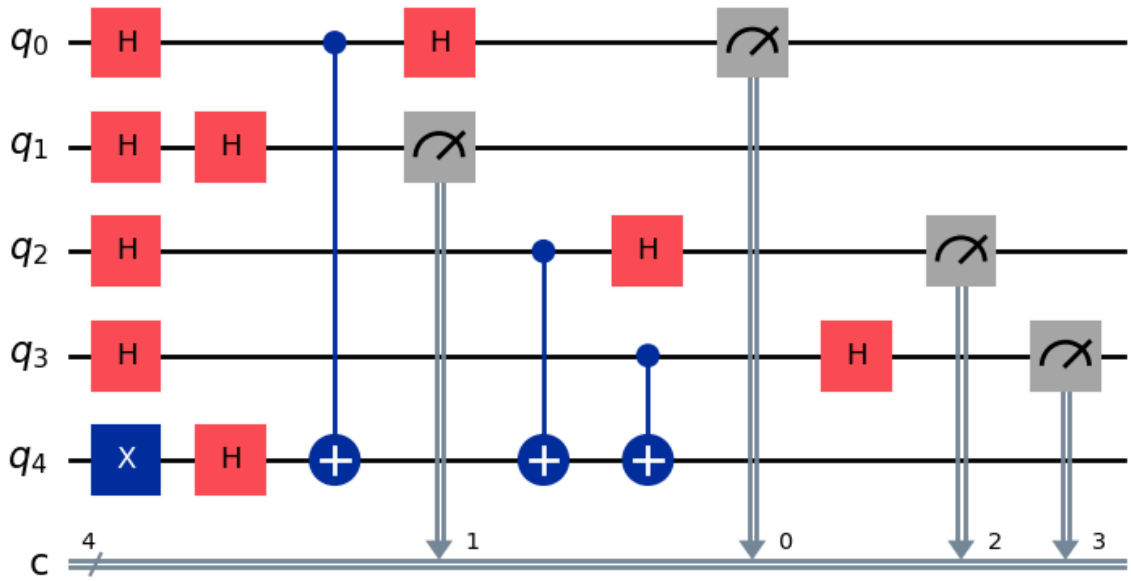


En este caso el string oculto es 0110 significa que solo dos bits del string son 1, así que el oráculo colocará dos compuertas CNOT hacia el qubit auxiliar. Todo lo demás en el algoritmo permanece igual. Después de la segunda capa de Hadamards, el estado final del registro principal debe corresponder a $|0110\rangle$, por lo que al medir se espera recuperar ese mismo string.

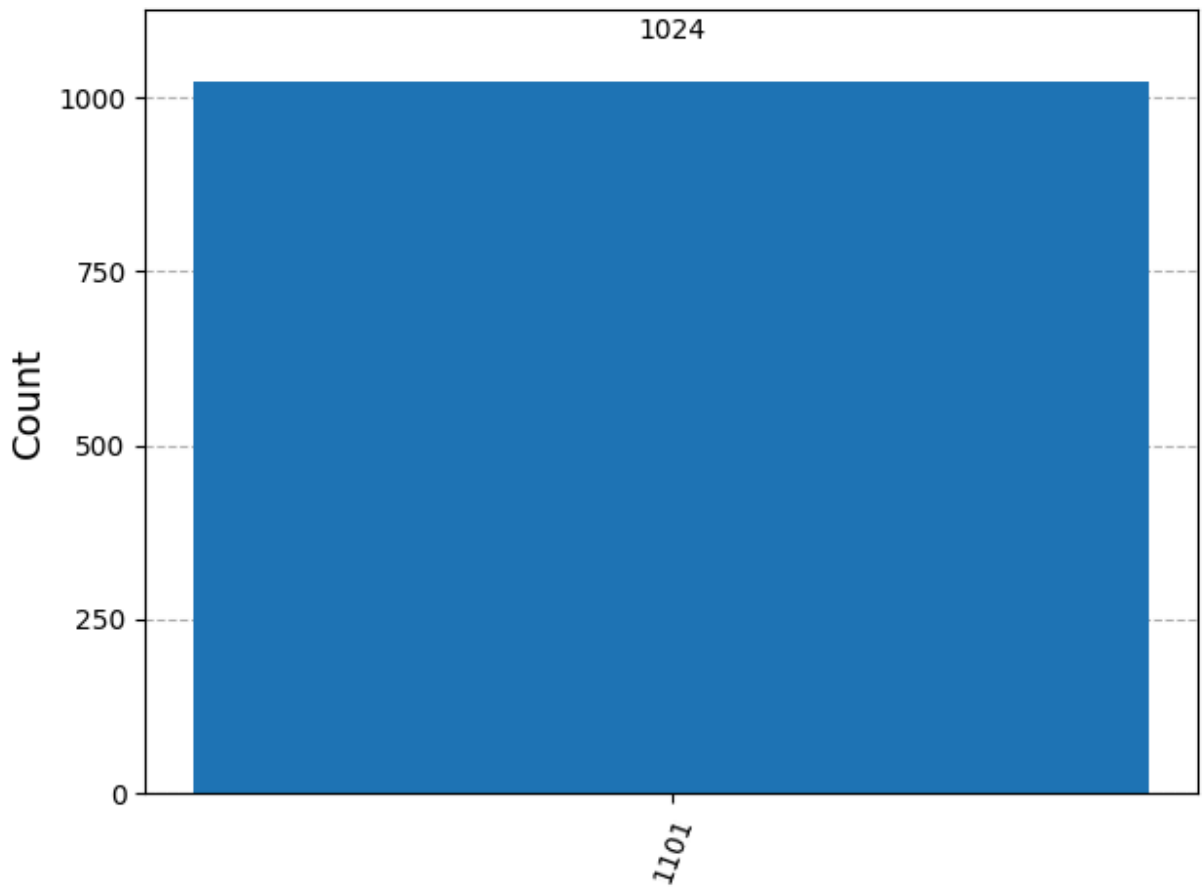
Caso: s=1101

```
qc_1101, counts_1101 = ejecutar_caso_bv("1101", shots=1024)
```

```
Circuito para s = 1101
```




```
Resultados para s = 1101: {'1101': 1024}
```

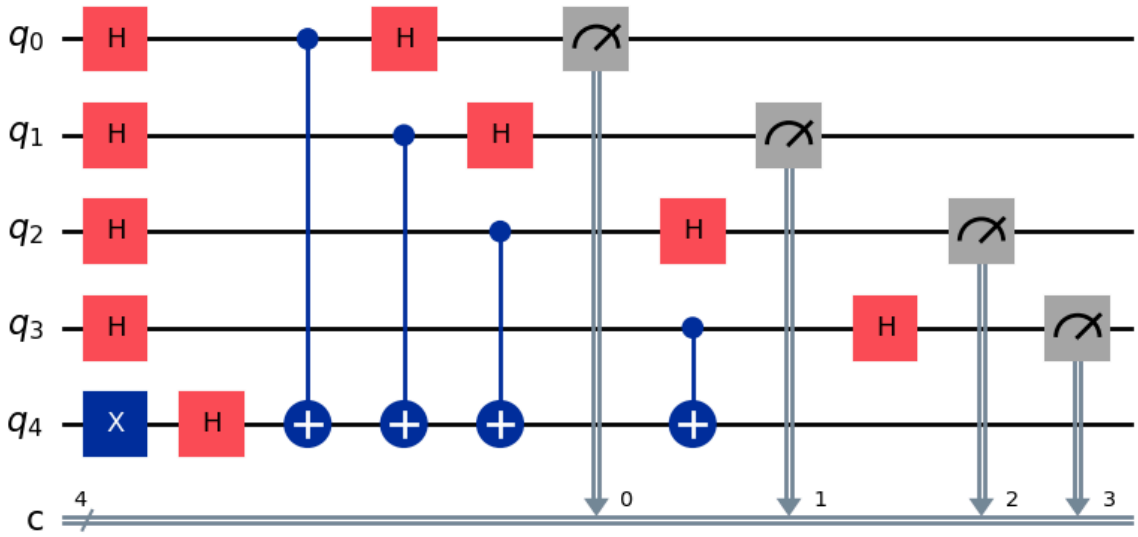


Ahora el string oculto es 1101 y el procedimiento no cambia. Lo único distinto es qué posiciones del string tienen valor 1, porque esas son las que determinan dónde se ponen las compuertas CNOT del oráculo. Al ejecutar el circuito, la medición debe concentrarse en el resultado 1101.

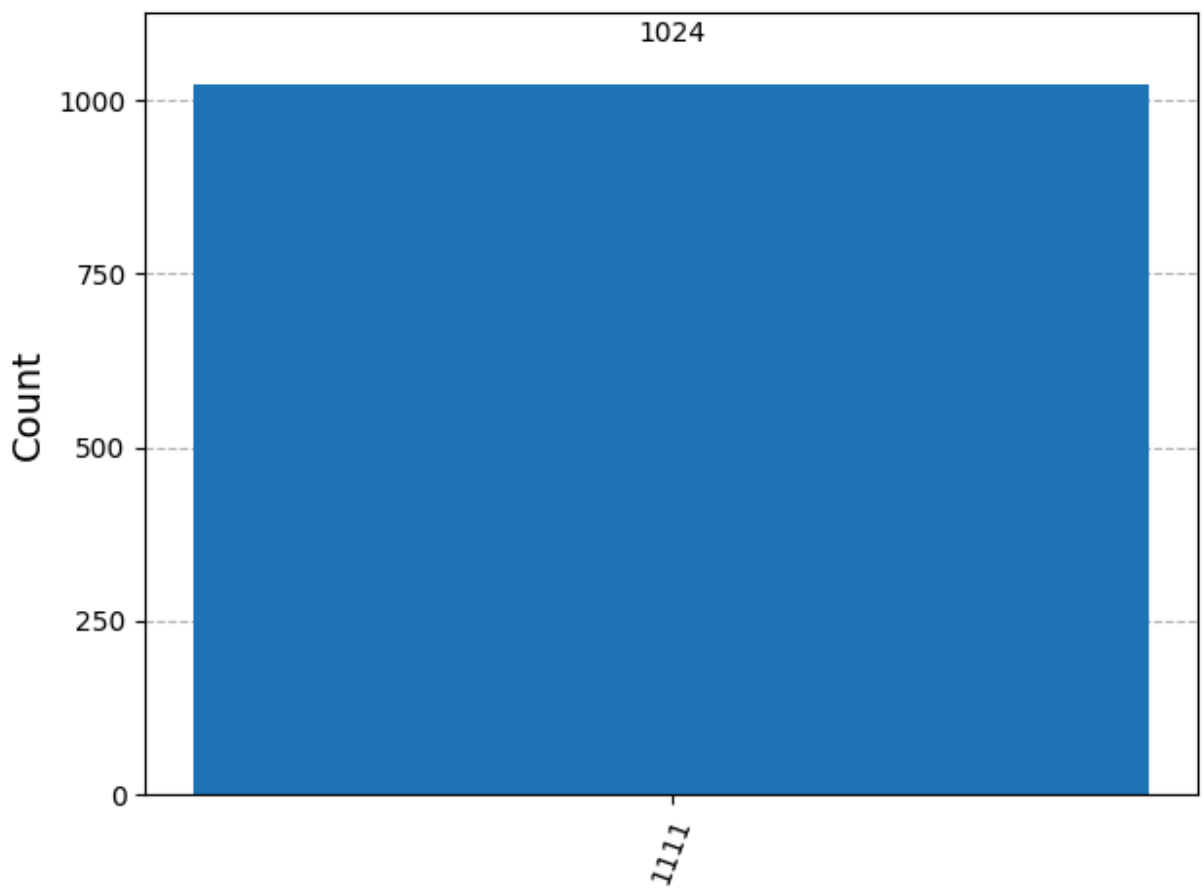
Caso: s=1111

```
qc_1111, counts_1111 = ejecutar_caso_bv("1111", shots=1024)
```

```
Circuito para s = 1111
```




```
Resultados para s = 1111: {'1111': 1024}
```

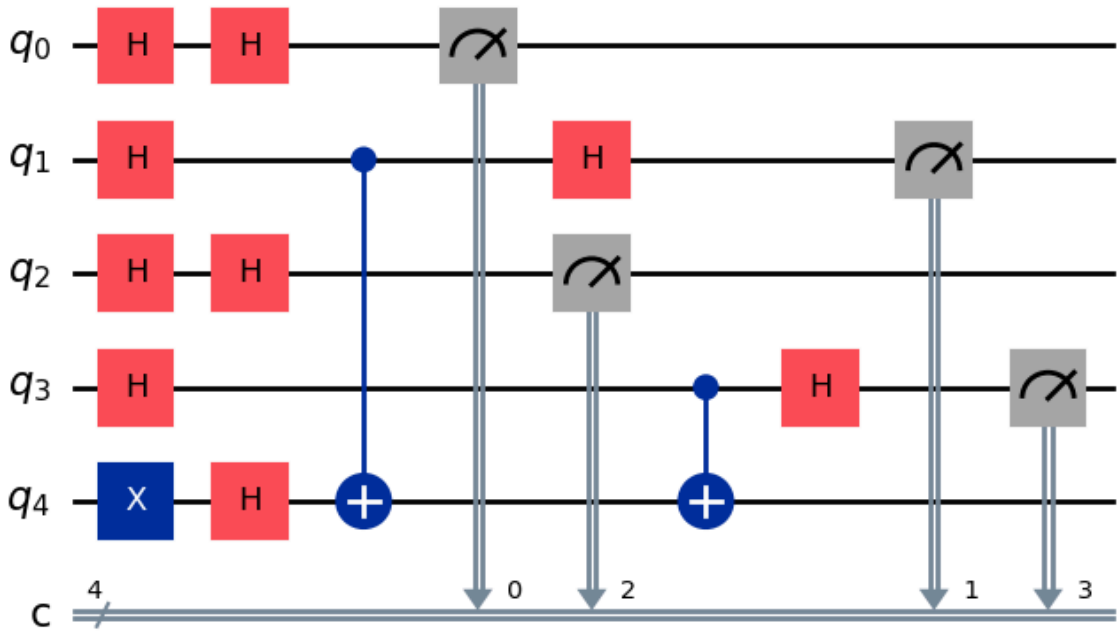


En este caso todos los bits del string son 1. Por lo tanto, el oráculo colocará una compuerta CNOT desde cada qubit del registro de entrada hacia el qubit auxiliar. Nuevamente, la estructura del algoritmo es exactamente la misma. Como el string oculto es 1111, eso es lo que debe aparecer al final en la medición.

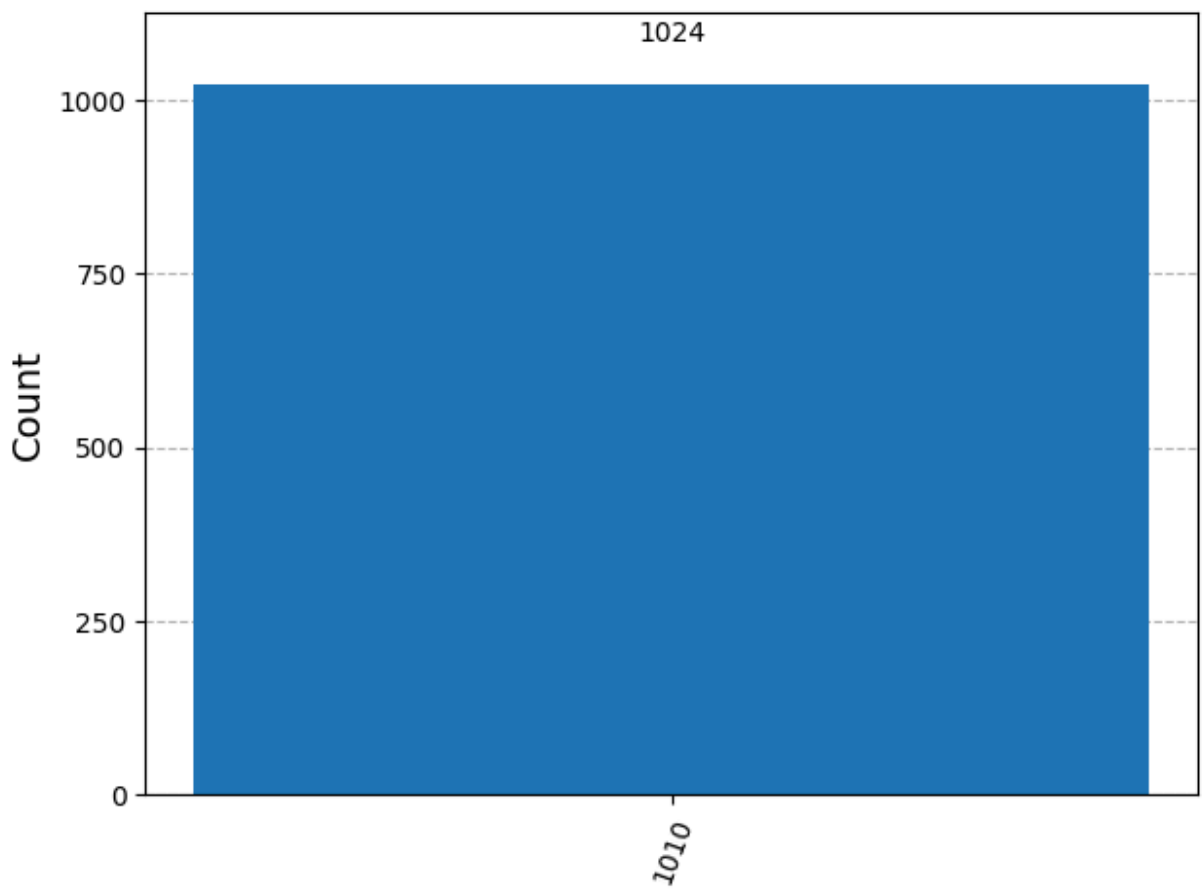
Caso 4: s=1010

```
qc_1010, counts_1010 = ejecutar_caso_bv("1010", shots=1024)
```

```
Circuito para s = 1010
```




```
Resultados para s = 1010: {'1010': 1024}
```



Finalmente, para 1010, el circuito sigue el mismo patrón. Solo cambian las posiciones donde se insertan las CNOT del oráculo. Este ejemplo confirma que el algoritmo no depende de un string específico, sino de la misma construcción general. La medición esperada debe ser 1010.

Comparación final de todos los casos:

```
resultados = {
    "0110": counts_0110,
    "1101": counts_1101,
    "1111": counts_1111,
    "1010": counts_1010
}

for s, counts in resultados.items():
    print(f"{s} -> {counts}")
```

```
0110 -> {'0110': 1024}
1101 -> {'1101': 1024}
1111 -> {'1111': 1024}
1010 -> {'1010': 1024}
```

Por lo tanto:

Para cada uno de los strings $s = 0110$, $s = 1101$, $s = 1111$ y $s = 1010$, el algoritmo de Bernstein–Vazirani se implementó siguiendo exactamente la misma estructura. Primero se preparó el qubit auxiliar en el estado $|1\rangle$, después se aplicaron compuertas Hadamard a todos los qubits y se construyó manualmente el oráculo sin usar la función `bv_oracle(n,s)`. Este oráculo se armó colocando compuertas CNOT únicamente en las posiciones donde el string oculto tiene bits iguales a 1, de modo que se implementa la función $f_s(x) = s \cdot x \pmod 2$. Luego se aplicaron nuevamente compuertas Hadamard al registro de entrada y se realizó la medición. En todos los casos, el resultado dominante coincidió con el string oculto correspondiente, lo que confirma que el algoritmo recupera correctamente s en una sola ejecución ideal del circuito.

2. Simon (b)

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

Utilidades

```
def xor_bits(a, b):
    return ''.join(str(int(x) ^ int(y)) for x, y in zip(a, b))

def dot_mod2(a, b):
    return sum(int(x) * int(y) for x, y in zip(a, b)) % 2

def rank_mod2(rows):
    A = [list(map(int, r)) for r in rows if r != "0000"]
    if not A:
        return 0
    m = len(A)
    n = len(A[0])
    r = 0
    c = 0
    while r < m and c < n:
        piv = None
        for i in range(r, m):
            if A[i][c] == 1:
                piv = i
                break
        if piv is None:
            c += 1
            continue
        A[r], A[piv] = A[piv], A[r]
        for i in range(m):
            if i != r and A[i][c] == 1:
                A[i] = [(x ^ y) for x, y in zip(A[i], A[r])]
        r += 1
        c += 1
    return r

def solve_simon_system(equations, n=4):
    eqs = [e for e in equations if e != "0000"]
    candidates = []
    for i in range(1, 2**n):
        s = format(i, f"0{n}b")
        ok = True
        for y in eqs:
            if dot_mod2(y, s) != 0:
                ok = False
                break
        if ok:
            candidates.append(s)
    return candidates
```

Tabla 2-a-1

```
def build_two_to_one_table(s):
    n = len(s)
    used = set()
    reps = []
    for i in range(2**n):
        x = format(i, f"0{n}b")
        x2 = xor_bits(x, s)
        if x in used or x2 in used:
            continue
        reps.append(x)
        used.add(x)
        used.add(x2)
    labels = [format(i, f"0{n}b") for i in range(len(reps))]
    table = {}
    for rep, label in zip(reps, labels):
        pair = xor_bits(rep, s)
        table[rep] = label
        table[pair] = label
    return table

def verify_two_to_one(table, s):
    ok_pair = True
    for x in table:
        if table[x] != table[xor_bits(x, s)]:
            ok_pair = False
            break
    buckets = {}
    for x, y in table.items():
        buckets.setdefault(y, []).append(x)
    ok_two_to_one = (len(buckets) == 8) and all(len(v) == 2 for v in buckets.va
    return ok_pair and ok_two_to_one
```

Oráculo

```
def apply_x_mask_for_string(qc, reg, bitstring):
    n = len(bitstring)
    for j, bit in enumerate(bitstring):
        if bit == "0":
            qc.x(reg[n - 1 - j])

def controlled_write_output(qc, in_reg, out_reg, x, y):
    n = len(x)
    apply_x_mask_for_string(qc, in_reg, x)
    controls = [in_reg[i] for i in range(n)]
    for j, bit in enumerate(y):
        if bit == "1":
            qc.mcx(controls, out_reg[n - 1 - j])
    apply_x_mask_for_string(qc, in_reg, x)
```

```
def simon_oracle_from_table(table, n=4):
    xreg = QuantumRegister(n, "x")
    yreg = QuantumRegister(n, "y")
    qc = QuantumCircuit(xreg, yreg, name="Uf")
    for x in sorted(table.keys()):
        y = table[x]
        controlled_write_output(qc, xreg, yreg, x, y)
    return qc
```

Circuito

```
def build_simon_circuit_from_table(table, n=4):
    xreg = QuantumRegister(n, "x")
    yreg = QuantumRegister(n, "y")
    creg = ClassicalRegister(n, "c")
    qc = QuantumCircuit(xreg, yreg, creg)
    qc.h(xreg)
    oracle = simon_oracle_from_table(table, n)
    qc.compose(oracle, qubits=list(range(2 * n)), inplace=True)
    qc.h(xreg)
    qc.measure(xreg, creg)
    return qc
```

Ejecución

```
simulator = AerSimulator()

def one_shot_y(table, n=4):
    qc = build_simon_circuit_from_table(table, n)
    job = simulator.run(qc, shots=1)
    counts = job.result().get_counts()
    y = list(counts.keys())[0]
    return y, qc

def recover_s_with_simon(table, n=4, max_tries=60):
    equations = []
    eq_set = set()
    counts_all = {}
    last_qc = None

    for _ in range(max_tries):
        y, qc = one_shot_y(table, n)
        last_qc = qc

        counts_all[y] = counts_all.get(y, 0) + 1

        if y not in eq_set:
```

```
                eq_set.add(y)
                equations.append(y)

            candidates = solve_simon_system(equations, n=n)

            if len(candidates) == 1:
                recovered = candidates[0]
                return recovered, equations, counts_all, last_qc

        candidates = solve_simon_system(equations, n=n)
        recovered = candidates[0] if len(candidates) == 1 else None
        return recovered, equations, counts_all, last_qc
```

Casos

```
secrets = ["1010", "1101", "1111"]
tables = {s: build_two_to_one_table(s) for s in secrets}
```

Verificación

```
print("VERIFICACIÓN DE ORÁCULOS\n")
for s in secrets:
    ok = verify_two_to_one(tables[s], s)
    print(f"s = {s} -> 2-a-1 válido: {ok}")
```

```
VERIFICACIÓN DE ORÁCULOS

s = 1010 -> 2-a-1 válido: True
s = 1101 -> 2-a-1 válido: True
s = 1111 -> 2-a-1 válido: True
```

Resultados

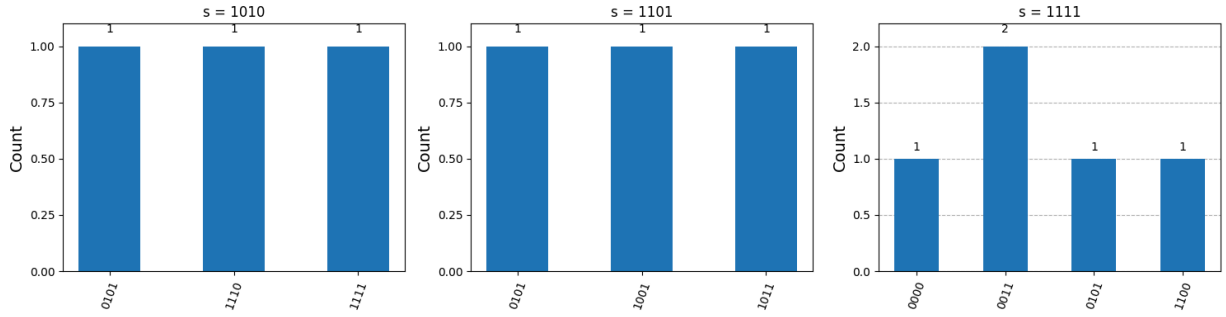
```
results = {}

for s in secrets:
    recovered, equations, counts_all, qc = recover_s_with_simon(tables[s], n=4)

    results[s] = {
        "recovered": recovered,
        "equations": equations,
        "counts": counts_all
    }

    print(f"--- Caso s = {s} ---")
    print("Mediciones acumuladas:", counts_all)
    print("Ecuaciones independientes (y):")
```

```
    for eq in equations:
        print(" ", eq)
    print("Candidatos para s:", solve_simon_system(equations, n=4))
    print("String recuperado:", recovered)
    print("Rango del sistema:", rank_mod2(equations))
    print()
```

```
--- Caso s = 1010 ---
Mediciones acumuladas: {'1011': 3, '1111': 1, '0000': 1, '0101': 1}
Ecuaciones independientes (y):
  1011
  1111
  0000
  0101
Candidatos para s: ['1010']
String recuperado: 1010
Rango del sistema: 3

--- Caso s = 1101 ---
Mediciones acumuladas: {'1001': 2, '0000': 1, '0101': 2, '0111': 1}
Ecuaciones independientes (y):
  1001
  0000
  0101
  0111
Candidatos para s: ['1101']
String recuperado: 1101
Rango del sistema: 3

--- Caso s = 1111 ---
Mediciones acumuladas: {'1010': 1, '1100': 1, '1001': 1}
Ecuaciones independientes (y):
  1010
  1100
  1001
Candidatos para s: ['1111']
String recuperado: 1111
Rango del sistema: 3
```

Histogramas

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, s in zip(axes, secrets):
    plot_histogram(results[s]["counts"], ax=ax)
    ax.set_title(f"s = {s}")

plt.tight_layout()
plt.show()
```



2. Simon (d)

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

Utilidades

```
def xor_bits(a, b):
    return ''.join(str(int(x) ^ int(y)) for x, y in zip(a, b))

def dot_mod2(a, b):
    return sum(int(x) * int(y) for x, y in zip(a, b)) % 2
```

Tablas de funciones 1-a-1 y 2-a-1

```
def build_one_to_one_table_identity(n=4):
    table = {}
    for i in range(2**n):
        x = format(i, f"0{n}b")
        table[x] = x
    return table
```

```
def build_one_to_one_table_reverse(n=4):
    table = {}
    for i in range(2**n):
        x = format(i, f"0{n}b")
        table[x] = x[::-1]
    return table

def build_two_to_one_table(s):
    n = len(s)
    used = set()
    reps = []
    for i in range(2**n):
        x = format(i, f"0{n}b")
        x2 = xor_bits(x, s)
        if x in used or x2 in used:
            continue
        reps.append(x)
        used.add(x)
        used.add(x2)

    labels = [format(i, f"0{n}b") for i in range(len(reps))]
    table = {}
    for rep, label in zip(reps, labels):
        pair = xor_bits(rep, s)
        table[rep] = label
        table[pair] = label
    return table

def is_one_to_one(table):
    values = list(table.values())
    return len(set(values)) == len(values)

def is_two_to_one(table):
    buckets = {}
    for x, y in table.items():
        buckets.setdefault(y, []).append(x)
    return all(len(v) == 2 for v in buckets.values())
```

Oráculo

```
def apply_x_mask_for_string(qc, reg, bitstring):
    n = len(bitstring)
    for j, bit in enumerate(bitstring):
        if bit == "0":
            qc.x(reg[n - 1 - j])

def controlled_write_output(qc, in_reg, out_reg, x, y):
    n = len(x)
    apply_x_mask_for_string(qc, in_reg, x)
```

```
    controls = [in_reg[i] for i in range(n)]
    for j, bit in enumerate(y):
        if bit == "1":
            qc.mcx(controls, out_reg[n - 1 - j])
    apply_x_mask_for_string(qc, in_reg, x)

def simon_oracle_from_table(table, n=4):
    xreg = QuantumRegister(n, "x")
    yreg = QuantumRegister(n, "y")
    qc = QuantumCircuit(xreg, yreg, name="Uf")
    for x in sorted(table.keys()):
        y = table[x]
        controlled_write_output(qc, xreg, yreg, x, y)
    return qc
```

Simon circuito

```
def build_simon_circuit(table, n=4):
    xreg = QuantumRegister(n, "x")
    yreg = QuantumRegister(n, "y")
    creg = ClassicalRegister(n, "c")
    qc = QuantumCircuit(xreg, yreg, creg)

    qc.h(xreg)
    oracle = simon_oracle_from_table(table, n)
    qc.compose(oracle, qubits=list(range(2 * n)), inplace=True)
    qc.h(xreg)
    qc.measure(xreg, creg)

    return qc
```

Ejecución

```
simulator = AerSimulator()

def run_simon_counts(table, shots=2048, n=4):
    qc = build_simon_circuit(table, n)
    job = simulator.run(qc, shots=shots)
    counts = job.result().get_counts()
    return qc, counts
```

Def. de los 4 oráculos

```
oracle_specs = {
    "1-a-1 identidad": build_one_to_one_table_identity(4),
    "1-a-1 reversa": build_one_to_one_table_reverse(4),
    "2-a-1 con s=1010": build_two_to_one_table("1010"),
```

```
    "2-a-1 con s=1111": build_two_to_one_table("1111"),
}
```

Verificación del tipo de función

```
print("VERIFICACIÓN DE LOS ORÁCULOS\n")
for name, table in oracle_specs.items():
    print(name)
    print("  Es 1-a-1:", is_one_to_one(table))
    print("  Es 2-a-1:", is_two_to_one(table))
    print()
```

```
VERIFICACIÓN DE LOS ORÁCULOS

1-a-1 identidad
  Es 1-a-1: True
  Es 2-a-1: False

1-a-1 reversa
  Es 1-a-1: True
  Es 2-a-1: False

2-a-1 con s=1010
  Es 1-a-1: False
  Es 2-a-1: True

2-a-1 con s=1111
  Es 1-a-1: False
  Es 2-a-1: True
```

Correr Simon para cada oráculo

```
results = {}

print("RESULTADOS DE LAS MEDICIONES\n")
for name, table in oracle_specs.items():
    qc, counts = run_simon_counts(table, shots=2048, n=4)
    results[name] = {"circuit": qc, "counts": counts}

    print(name)
    print("  Counts:", counts)
    print()
```

```
RESULTADOS DE LAS MEDICIONES

1-a-1 identidad
```